Phase 1

1A:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

sam@sam-Latitude-5430:~/Programming/PES2UG24CS437-pes-vcs$ make test_objects && ./test_objects
      |          ^~~~
```

```
sam@sam-Latitude-5430:~/Programming/PES2UG24CS437-pes-vcs$ make pes
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt …
/usr/include/x86_64-linux-gnu/bits/stdio2.h:54:10: note: '__builtin___snprintf_chk' output between 5 and 516 bytes into a destination of size 512
   54 |   return __builtin___snprintf_chk (__s, __n, __USE_FORTIFY_LEVEL - 1,
      |          ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   55 |                                    __glibc_objsize (__s), __fmt,
      |                                    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   56 |                                    __va_arg_pack ());
      |                                    ~~~~~~~~~~~~~~~~
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  modified:   file1.txt
  modified:   file2.txt

Untracked files:
  untracked:  object.c
  untracked:  pes.h
  untracked:  tree.h
  untracked:  index.h
  untracked:  Makefile
  untracked:  .gitignore
  untracked:  tree.c
  untracked:  test_sequence.sh
  untracked:  test_objects.c
  untracked:  pes.c
  untracked:  test_tree
  untracked:  README.md
  untracked:  test_tree.c
  untracked:  bye.txt
  untracked:  hello.txt
  untracked:  index.c
  untracked:  commit.c
  untracked:  test_objects
  untracked:  commit.h

100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f file2.txt
```

3B:

```
sam@sam-Latitude-5430:~/Programming/PES2UG24CS437-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f file2.txt
```

Phase 4

4A:

```
● sam@sam-Latitude-5430:~/Programming/PES2UG24CS437-pes-vcs$ ./pes log
  commit 0f072fbc1a4e290543598be07374812943313edcaaf6934288175731fbb10813
  Author: PES User <pes@localhost>
  Date:   1776347006

      Add farewelldd f

  commit 625a5593ab1058f26ba3c8ca0577df796d57c72445cd3769a3928785a1d22b79
  Author: PES User <pes@localhost>
  Date:   1776347006

      Add worldelldd f

  commit 3b50807d3998a9e2662e586a8f51fefe9059736f062447b2b8c90bf74330bc93
  Author: PES User <pes@localhost>
  Date:   1776347006

      Initial commit
```

4B:

```
● sam@sam-Latitude-5430:~/Programming/PES2UG24CS437-pes-vcs$ find .pes -type f | sort
  .pes/HEAD
  .pes/index
  .pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
  .pes/objects/0f/072fbc1a4e290543598be07374812943313edcaaf6934288175731fbb10813
  .pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
  .pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
  .pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
  .pes/objects/3b/50807d3998a9e2662e586a8f51fefe9059736f062447b2b8c90bf74330bc93
  .pes/objects/62/5a5593ab1058f26ba3c8ca0577df796d57c72445cd3769a3928785a1d22b79
  .pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
  .pes/objects/85/e367b76c2417b47a95f46208835ad8700dd58335171c94a497974be6191900
  .pes/objects/96/c0d5c28dd3b719606c36723cd16aefbc73b3dc8b74de61070183357d6968b9
  .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
  .pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
  .pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
  .pes/objects/f1/0239b620372163c2dd06ac33189180c55edf1c56b15990a1cf44e00b1b0e59
  .pes/refs/heads/main
```

4C:

```
● sam@sam-Latitude-5430:~/Programming/PES2UG24CS437-pes-vcs$ cat .pes/refs/heads/main
  0f072fbc1a4e290543598be07374812943313edcaaf6934288175731fbb10813
● sam@sam-Latitude-5430:~/Programming/PES2UG24CS437-pes-vcs$ cat .pes/HEAD
  ref: refs/heads/main
○ sam@sam-Latitude-5430:~/Programming/PES2UG24CS437-pes-vcs$
```

test-integration:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
sam@sam-Latitude-5430:~/Programming/PES2UG24CS437-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  modified:   file.txt
  modified:   hello.txt

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 64504080325e... Initial commit

Log after first commit:
commit 64504080325eda5b8fb82203f53d780aa8ddd5a727679bf45b11c58bf8849830
Author: PES User <pes@localhost>
Date:   1776356142

    Initial committial commiQ


--- Second Commit ---
Committed: 8db65ae5f61d... Update file.txt

--- Third Commit ---
Committed: 9505b478d91b... Add farewell

--- Full History ---
commit 9505b478d91b48b158907b5224280aec6ca61c0f353aee25399280a5951e5bdc
Author: PES User <pes@localhost>
Date:   1776356143
```

```
    Add farewelldd f

commit 8db65ae5f61dba5682da99b8957d3cf9abcc820cae8c5a03e91bccce44d357bf
Author: PES User <pes@localhost>
Date:   1776356143

    Update file.txtf

commit 64504080325eda5b8fb82203f53d780aa8ddd5a727679bf45b11c58bf8849830
Author: PES User <pes@localhost>
Date:   1776356142

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
9505b478d91b48b158907b5224280aec6ca61c0f353aee25399280a5951e5bdc

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/64/504080325eda5b8fb82203f53d780aa8ddd5a727679bf45b11c58bf8849830
.pes/objects/8d/b65ae5f61dba5682da99b8957d3cf9abcc820cae8c5a03e91bccce44d357bf
.pes/objects/95/05b478d91b48b158907b5224280aec6ca61c0f353aee25399280a5951e5bdc
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
sam@sam-Latitude-5430:~/Programming/PES2UG24CS437-pes-vcs$
```

Phase 5 - Branching and Checkout

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

The first thing to do here is to make HEAD point to the commit hash of the new branch. This is simple, all you have to do is find the branch's ref in .pes/refs/heads/ and then call head_update() with it. The problem is updating the current working directory. Now you have to take the tree that this commit has and then actually create the directory from it. And you have to make sure that you're not overwriting unsaved changes from the previous branch yet as well - you have to detect that such changes are present and then abort the checkout.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between

branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

So there's 2 main conditions you have to check. The first thing is, do you have any uncommited changes on your current branch? To check this, you hash all the files in your working directory and then compare with the hashes in the index. If they are different, then unsaved changes do exist.

However, this alone is not enough to abort the checkout because when switching branches, git only overwrites files which have different contents in the current branch and the new branch. So you're only at risk of losing work if git detects that the contents in the file are different between the two branches and wants to overwrite. To check this condition, you have to take the hashes in the index of the tree that you're branching to and compare with your current branch's index's hashes. If they differ, then you know a change has been made.

If BOTH conditions are true, then there is risk of work being lost. Hence, abort the checkout.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

If you make commits in this state, these commits will not be saved to a certain branch at all. The commits would be chained along in .pes/HEAD. Hence when you switch back to the main branch, you will lose all your commits because the commit stored in .pes/HASH would be overwritten. This is intentionally designed by git to allow you to experiment freely without creating a mess in your commit history.

Currently, pes-vcs doesn't have a mechanism to recover the lost commits from switching back from detached mode to attached head mode, but it could easily be implemented by saving the pes/HEAD commit hash in a file, maybe .pes/detached_head, and then switching branches (i.e. overwriting pes/HEAD to have a reference instead of a commit hash directly).

Phase 6 - Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

You can use BFS or DFS for this. To keep track of reachable hashes, you should use a set because searching in a set is O(1) due to hashing. Basically, you will start with the latest commits on each branch (you can find the hashes for these in .pes/refs/heads), add it to the set of reachable hashes, build the tree from it, and then add the tree's subtrees to the stack/queue. You will continue this process until you hit a blob (it's the leaf of the tree, you can't traverse anywhere further, so add nothing to the stack/queue). Continue until the stack/queue is empty. Repeat this for all the branches, and then delete whatever object hashes are not present in the set from .pes/objects.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Objects are not created on commit, they are created whenever the index is updated. Hence, after staging some items, they will exist in the object store.

Consider a scenario where some new files were staged. These objects exist in the object store. A commit is initiated to save these files, and at the same time, garbage collection is running. Because the garbage collector can't see the new commit yet, it will traverse through the older commit's files and then decide that the new objects that were just created are not reachable and will delete them. Then the commit would also become corrupted because the objects that are being committed don't exist anymore.

To prevent this, git does a couple things. Firstly, it has reflog. Reflog is basically tracking recently used objects. Even if an object is unreachable, if it has been created or modified recently, then git will conclude it's not safe to delete and leave it. Secondly, to prevent temporary objects being created during committing from being deleted prematurely, the branch where the commit is happening is "locked". Git will see the state before the commit was initiated, and then after the commit is over. It will never see the temp files.